

Quantum-Ready Evidence Graphs and QPU-Factor Evaluation 1.0

Classical Claim-Selection Baselines, Simulator Boundaries, and a Non-Production Measurement-Factor Contract

Shafaet Brady Hussain | Independent researcher, ResearchForumOnline | 18 August 2026

Abstract

I consolidate two related but distinct engineering questions: how to select a defensible subset of claims from a provenance graph, and how a remote measurement record could be mixed with a local random nonce without misdescribing the result as quantum encryption. The release supplies an eight-node synthetic claim graph, a constrained objective, exact enumeration of all 256 subsets, and a deterministic greedy comparator. It also audits an existing JavaScript QPU-factor contract and test. That test passes, but its hardware-shaped evidence object is fabricated. It therefore demonstrates schema validation, commitment checking, HKDF domain separation, tamper rejection, and a default simulator-rejection boundary—not quantum-hardware execution, certified entropy, provider validation, quantum security, or quantum advantage. “Quantum-ready” means the optimization problem and evidence interface are explicit enough for a future experiment to be compared with a classical reference. No quantum credits were spent.

1. Research questions and contributions

This paper asks: (1) can claim selection be encoded as a small, auditable constrained optimization problem with an exact classical reference; and (2) what does the present QPU-factor implementation actually establish?

I specify a graph containing claims, review attributes, dependencies, contradictions, and support relations; release exact and greedy classical solvers; separate cryptographic-contract evidence from hardware or entropy evidence; and publish falsifiable gates for future simulator or hardware work. The earlier working draft proposed a source-ledger-to-claim-graph workflow. This version consolidates that proposal with executable evidence and does not preserve broad speculative claims as findings.

2. Evidence-graph model

Let each claim node i have utility u_i , review cost c_i , and claim risk r_i . For selected set S , the released objective is:

$$F(S) = \sum_{i \in S} (u_i - \lambda r_i) + \beta * \text{supported_pairs}(S).$$

Constraints require total cost not to exceed budget B , each selected dependent claim to include its prerequisite, and mutually contradictory claims not to be selected together. In the release, $\lambda = 0.65$, $\beta = 1.25$, $B = 11$, and there are eight claims. These are synthetic test parameters, not empirically estimated editorial weights.

The data model is provenance-aware: a claim can be linked to entity, activity, or agent records rather than storing provenance only as prose. PROV-O supplies an interoperable vocabulary for such descriptions [1]. This release does not claim full PROV-O conformance; conversion and validation remain future work.

3. Reproducible classical evaluation

The reference evaluator enumerates $2^8 = 256$ subsets, rejects infeasible selections, and chooses the maximum score with deterministic tie-breaking. The greedy comparator ranks nodes by risk-adjusted utility per cost and adds a node only when the partial set remains feasible. Running `python verify_summary.py` regenerates and checks the result file.

This benchmark is deliberately small. Its purpose is to give any future heuristic—including QAOA-inspired optimization—a known optimum. Farhi, Goldstone, and Gutmann introduced QAOA as an approximate quantum algorithm for combinatorial optimization [2]. Citing it does not make this computation quantum: every optimization result reported here is classical.

4. Meaning of “quantum-ready”

“Quantum-ready” is an interface and evaluation property. Variables, objective, constraints, instance, exact optimum, and output schema are explicit. A future candidate can therefore be scored for objective quality, constraint violations, runtime, sampling variance, cost, and rerun reproducibility against the same instance.

It does not mean a quantum processor has read documents, inferred truth, improved claims, or achieved advantage. A credible future comparison would pre-register the circuit or mapping, provider/backend class, shot budget, seeds where applicable, classical optimizer, calibration context, and acceptance thresholds. Simulator results must remain labelled as simulator results. Hardware execution would prove only that a specified job ran and produced recorded measurements; it would not validate the document claims.

5. QPU-factor contract under test

The inspected browser module accepts a 32-byte local nonce and an evidence object with provider, API, backend, source class, status, UUID, hashes, qubits, and shots. It checks that $\text{SHA-256}(\text{local nonce})$ equals the client commitment. It derives a 256-bit factor with HKDF-SHA-512, using the measurement digest as salt and domain-separated context, then emits a factor commitment and evidence digest.

HKDF is an extract-and-expand key-derivation function. RFC 5869 defines its construction and explains the role of salt; salt is not a substitute for input keying material [3]. The module defaults to requiring a hardware source class, so a simulator record is rejected unless the caller explicitly relaxes the requirement.

The automated test passes derivation, evidence-binding, tamper, and simulator-boundary cases. However, its hardware-looking fixture—backend label, job identifier, and repeated hashes—is fabricated. The result is a software-contract test only.

The local cryptographically secure random nonce is the security-critical input. A measurement digest used as HKDF salt can bind derivation to a record, but it does not prove that the record contains entropy and does not create entropy by itself. Entropy claims require a source model and assessment; NIST SP 800-90B describes requirements and tests for entropy sources used for random-bit generation [4]. This release performs no such assessment.

6. Threat analysis

The contract provides useful failure boundaries: altered evidence or factor data is rejected; a nonce commitment detects mismatch; domain separation reduces cross-protocol reuse; and simulator provenance is rejected by default. These are implementation-and-test properties.

The contract does not establish authenticity of a provider record unless signatures or independently verified retrieval bind it to a trusted source. A malicious endpoint could fabricate internally consistent fields. Replayed measurements, compromised local randomness, ambiguous canonicalization, dependency substitution, downgrade of `requireHardware`, and incorrect caller handling remain threats. The factor is not a new base cipher; it is an optional derivation input around conventional primitives.

7. Unexecuted hardware evaluation protocol

A July 2026 technical note proposed, rather than reported, an eight-week evaluation using 120 iterations and 48,000 shots. Planned measures included total-variation distance, bit bias, mutual information, conservative min-entropy estimates, drift, and tamper/rejection behavior. It stated that the current prototype was not “quantum encrypted” and that plaintext, final keys, and the local nonce should not be sent to a provider.

That plan remains unexecuted here. No provider is named as an endorser, no credit is consumed, and no hardware result is reported. Before execution, the experiment needs written scope, budget confirmation, a frozen protocol, authenticated evidence format, and publication wording appropriate to provider terms.

8. Falsifiable claims and release gates

1. The supplied verifier evaluates exactly 256 subsets; failure falsifies the reproducibility claim.
2. The exact result is at least as good as the greedy result; a ratio above 1 falsifies the implementation.
3. The QPU-factor test passes on the identified source revision; failure falsifies the reported contract result.
4. A simulator record is rejected under the default hardware-required path; acceptance falsifies the stated boundary.
5. No future quantum result should be released without raw job/evidence record, hash, source classification, classical reference, cost, and rerun protocol.
6. No entropy claim should be made until a source model and appropriate entropy assessment are published.

9. Limitations

The graph is synthetic and small. Its weights are illustrative, the greedy method is simple, and exact enumeration does not scale. The result demonstrates reproducibility of an encoding, not truth selection, scholarly quality, or real editorial preference. There is no human-subject study, external reviewer study, production benchmark, hardware run, entropy certification, security audit, peer review, or independent validation. The format is not yet PROV-O conformant. The QPU-factor code was evaluated through existing tests, not a formal proof or external penetration test.

10. Conclusion

The defensible result is modest and useful: a claim graph can be reduced to an explicit constrained objective and checked against an exact classical baseline, while a measurement-factor interface can be tested without conflating a synthetic fixture with quantum evidence. This creates a clean departure point for later simulator or hardware work. Quantum advantage, certified randomness, provider validation, and quantum-secure encryption remain outside the evidence.

Reproducibility and evidence ledger

- Paper source: `papers/quantum-ready-evidence-graphs.md`.
- Classical package: `artifacts/quantum-evidence-qpu-factor-1.0/`.
- QPU-factor module SHA-256:
`336b3fde202ae4a62087e889636ac4d8d7383e0510c2bbb46e139e86599e7787`.
- QPU-factor test SHA-256:
`0d35a373a2e68ec29b0a5bb3f56d90bb9c0dd53ae9e3ad63c54d8ef05f69f120`.
- Planned-protocol note SHA-256:
`d8cdda02286fe113818af086695706b1f741dbfabed827d52b6552df167c7d6e`.
- QPU-factor test result: pass; fixture synthetic; no hardware evidence.

AI-use disclosure

AI tools assisted corpus discovery, code inspection, drafting, editing, and packaging. I take responsibility for the paper's claims and release decisions. AI assistance is not independent validation or peer review.

Licensing

The paper is released under CC BY 4.0. The new classical reproducibility code is released under the MIT License. Referenced application code retains its original repository terms and is not redistributed in this package.

References

- [1] W3C. *PROV-O: The PROV Ontology*. W3C Recommendation, 2013. <https://www.w3.org/TR/prov-o/>
- [2] E. Farhi, J. Goldstone, and S. Gutmann. *A Quantum Approximate Optimization Algorithm*. arXiv:1411.4028, 2014. <https://arxiv.org/abs/1411.4028>
- [3] H. Krawczyk and P. Eronen. *HMAC-based Extract-and-Expand Key Derivation Function (HKDF)*. RFC 5869, 2010. <https://www.rfc-editor.org/rfc/rfc5869>
- [4] NIST. *Recommendation for the Entropy Sources Used for Random Bit Generation*. SP 800-90B, 2018. <https://csrc.nist.gov/pubs/sp/800/90/b/final>